Karlsruher Institut für Technologie

Institut für Informationssicherheit und Verlässlichkeit
Prof. Dr. André Platzer platzer@kit.edu
Prof. Dr. Ralf H. Reussner reussner@kit.edu
Prof. Dr. Ina Schaefer ina.schaefer@kit.edu

Klausur Programmierparadigmen — Beispiellösung

WS24/25, 27. März 2025, 08:00 – 10:00 Uhr

Bearbeitungszeit: 120 min

Aufgabe 1 (Allgemeine Fragen) [10 Punkte]

Kreuzen Sie die richtigen Antworten an. Es können beliebig viele Antwortmöglichkeiten richtig sein (mindestens jedoch 1). Jede vollständig richtige Antwort gibt einen (1) Punkt. Wenn Sie Ihre Antwort korrigieren möchten, malen Sie das Quadrat aus und setzen Sie das Kreuz neu oder malen Sie es neben ein ausgemaltes Feld.

Funktionale Programmierung

(a) Welche Aussagen gelten für die Auswertungsreihenfolge in Haskell?

- ☐ Ausdrücke die weiter vorne in der Datei stehen werden immer vor Ausdrücken die weiter hinten stehen ausgewertet
- ☐ Die Werte von Argumenten werden immer vor dem Funktionsrumpf ausgewertet
- ☒ Funktionale Programme können nichtterminierend sein
- ☐ Funktionale Programme können immer nur sequentiell nie parallel ausgewertet werden

(b) Was gilt dank Haskell's Lazy Evaluation?

- ☒ Die Faltung `foldr (&&) True` ist über manchen Listen schneller als lineare Zeit
- ☐ `(take 5) . sort` braucht für alle `sort` Implementierung nur 5 Vergleiche der Listenelemente
- ☐ `(take 5) . sort` braucht für alle `sort` Implementierung nur linear viele Vergleiche der Listenelemente
- ☐ Das Programm was nach der Zeile `ones = 1 : ones` steht ist unerreichbar

(c) Die Funktion `foldl (+) 1` wird ...

- ☒ Auf allen endlichen Integerlisten funktionieren
- ☐ Auf allen unendlichen Integerlisten funktionieren
- ☐ Auf endlichen Listen von endlichen Integerlisten funktionieren
- ☐ 6 zurückliefern für die Eingabe `[1,2,3]`

(d) Ein Aufruf der Funktion `\f -> f (3 + 1) == f (2 + 2)` kann...

- ☒ `True` zurückliefern
- ☐ `False` zurückliefern
- ☒ nichtterminierend sein
- ☒ einen Laufzeitfehler verursachen

Parallelprogrammierung

(e) Wie wird die von MPI mittels kollektiver Operationen realisierte Parallelität nach Flynn's Taxonomie klassifiziert?

- ☐ Single Instruction x Single Data
- ☒ Single Instruction x Multiple Data
- ☐ Multiple Instruction x Single Data
- ☐ Multiple Instruction x Multiple Data

(f) Welche Coffman-Bedingungen können bei der Nutzung von `synchronized`-Blöcken in Java vermieden werden?

- ☐ Mutual exclusion
- ☒ Hold and wait
- ☐ No preemption
- ☒ Circular wait

(g) Welche Bedingungen müssen bei Design-by-Contract nach Ausführung einer Methode erfüllt sein?

- ☐ Vorbedingung der Methode
- ☒ Klasseninvariante
- ☒ Nachbedingung der Methode
- ☒ Nachbedingung der überschriebenen Methode in der Superklasse

Compiler

(h) Warum können Syntheseattribute in einer attributierten Grammatik leichter von unten nach oben berechnet werden?

- ☒ Weil sie nur von Kindknoten abhängen
- ☐ Weil Syntheseattribute keine Abhängigkeiten haben
- ☒ Weil sie beim Bottom-up-Parsing direkt mit der Reduktion berechnet werden können
- ☐ Weil Syntheseattribute keine Rolle bei der Semantikanalyse spielen

(i) Warum reicht die Namensanalyse allein nicht aus, um alle semantischen Fehler in einem Programm zu erkennen?

- ☐ Weil sie keine Namen auflöst
- ☒ Weil sie keine Typüberprüfung durchführt
- ☒ Weil sie keine Gültigkeitsbereiche für Variablen überprüft
- ☐ Weil sie nur in interpretierten Sprachen notwendig ist

(j) Warum kann aggressive Inlining-Optimierung nachteilig sein?

- ☒ Sie kann zu einer explosionsartigen Codevergrößerung führen
- ☐ Sie verringert die Anzahl der Funktionsaufrufe nicht wirklich
- ☐ Sie führt immer zu höherem Speicherverbrauch zur Laufzeit
- ☒ Sie kann die Cache-Effizienz negativ beeinflussen

```
-- Erinnerung: Typsignaturen
map :: (a -> b) -> [a] -> [b]
filter :: (a -> Bool) -> [a] -> [a]
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
all :: Foldable t => (a -> Bool) -> t a -> Bool
any :: Foldable t => (a -> Bool) -> t a -> Bool
```

Aufgabe 2 (Haskell: Rucksack packen) [10 Punkte]

Sie planen eine Wanderung und wollen Ihren Rucksack packen. Leider hat Ihr Rucksack nur ein begrenztes Fassungsvermögen, sodass Sie abwägen müssen, welche Gegenstände Sie mitnehmen wollen. Sie haben eine Liste an Gegenständen, die jeweils einen Wert v_i und ein Gewicht w_i haben und einen Rucksack, der höchstens Gewicht W tragen kann. Ihr Ziel ist es, den Gesamtwert der Gegenstände, die Sie mitnehmen, zu maximieren. *Jeder Gegenstand kann höchstens einmal mitgenommen werden.*

```
type Value = Double
type Weight = Double
type Item = (Value, Weight)
```

Da Sie es eilig haben, entscheiden Sie sich für einen schnellen und gierigen Ansatz.

(a) Implementieren Sie eine Funktion [2 Punkte]

```
takeItem :: (Value,Weight) -> Item -> (Value,Weight)
```

die einen Gegenstand dem Rucksack hinzufügt sofern in diesem noch Platz ist. Der Rucksack ist gegeben durch den Gesamtwert der bereits enthaltenen Gegenstände und dem *verbleibenden* Gewicht.

Beispiellösung:

```
takeItem :: (Value,Weight) -> Item -> (Value,Weight)
takeItem (bagVal,bagWeight) (itemVal,itemWeight)
  | itemWeight <= bagWeight = (bagVal + itemVal, bagWeight - itemWeight)
  | otherwise = (bagVal, bagWeight)
```

(b) Implementieren Sie eine Funktion [4 Punkte]

```
sortItems :: [Item] -> [Item]
```

die Gegenstände *absteigend* nach Ihrem Wert/Gewicht-Verhältnis v_i/w_i sortiert. Sie können die Funktion `sort :: Ord a => [a] -> [a]` verwenden, die eine Liste in aufsteigender Reihenfolge sortiert.

Erinnerung:
Haskell ordnet Tupel lexikografisch: Zunächst wird nur das jeweils erste Element verglichen, bei Gleichheit folgt ein Vergleich des jeweils zweiten Elements usw.

Beispiel:
`sortItems [(10,2),(6,1),(12,3),(9,2)] = [(6,1),(10,2),(9,2),(12,3)]`

Beispiellösung:

```
sortItems :: [Item] -> [Item]
sortItems xs = reverse $ (map snd) $ sort $ [(ratio x, x) | x <- xs]
  where ratio (price, weight) = price / weight
```

(c) Implementieren Sie eine Funktion [4 Punkte]

```
greedyKnapsack :: [Item] -> Weight -> Value
```

die zunächst die Gegenstände nach ihrem Wert/Gewicht-Verhältnis sortiert und dann die Gegenstände der Reihe nach, entsprechend dieser Sortierung, auswählt, wenn sie in den Rucksack passen.

Beispiel:
`greedyKnapsack [(10,2),(6,1),(12,3)] 5 = 16.0`

Beispiellösung:

```
greedyKnapsack :: [Item] -> Weight -> Value
greedyKnapsack items w = fst (foldl takeItem (0,w) (sortItems items))
```

Aufgabe 3 (Haskell: Heapsort) [16 Punkte]

In dieser Aufgabe wollen wir Zahlen mittels eines Heaps sortieren.

Ein Heap (Haufen) ist ein Binärbaum der folgende Invariante erfüllt: Der Wert jedes Knoten ist kleiner als der seiner Kinderknoten.

Nachfolgend finden Sie den Typ für Heaps in Haskell:

```
data Heap t =
  Leaf |
  Node (Heap t) t (Heap t)
```

(a) Implementieren Sie eine Funktion [2 Punkte]

```
minDepth :: Heap t -> Integer
```

die die minimale Tiefe eines Heaps berechnet, d.h. die kürzeste Entfernung zu einem Blatt. Blattknoten haben Tiefe 0.

Beispiel:
`minDepth (Node (Node Leaf 1 Leaf) 0 Leaf) = 1`

Wir werden linksgerichtete Heaps betrachten, was bedeutet, dass `minDepth` für das rechte Kind niemals größer ist als für das linke Kind.

Beispiellösung:

```
minDepth :: Ord t => Heap t -> Integer
minDepth Leaf = 0
minDepth (Node left _ right) = 1 + min (minDepth left) (minDepth right)
-- alternativ (für linksgerichtete Heaps)
minDepth Leaf = 0
minDepth (Node _ _ right) = 1 + minDepth right
```

(b) Für die folgenden Aufgaben können Sie die Funktion [2 Punkte]

```
merge :: Ord t => Heap t -> Heap t -> Heap t
```

verwenden, die zwei linksgerichtete Heaps zu einem einzigen linksgerichteten Heap zusammenfügt. Implementieren Sie eine Funktion

```
insert :: Ord t => Heap t -> t -> Heap t
```

die ein Element zu einem linksgerichteten Heap hinzufügt.

Beispiellösung:

```
insert :: Ord t => Heap t -> t -> Heap t
insert h v = merge h (Node Leaf v Leaf)
```

(c) Implementieren Sie eine Funktion [3 Punkte]

```
extractHeap :: Ord t => Heap t -> [t]
```

die einen linksgerichteten Heap in eine sortierte Liste umwandelt.

Beispiellösung:

```
extractHeap :: Ord t => Heap t -> [t]
extractHeap Leaf = []
extractHeap (Node left v right) = v : (extractHeap (merge left right))
```

(d) Implementieren Sie eine Funktion [3 Punkte]

```
heapSort :: Ord t => [t] -> [t]
```

die eine Liste mittels eines linksgerichteten Heaps sortiert.

Beispiel:

```
heapSort [1,3,5,2,4] = [1,2,3,4,5]
```

Beispiellösung:

```
heapSort :: Ord t => [t] -> [t]
heapSort xs = extractHeap (foldl insert Leaf xs)
```

(e) Implementieren Sie nun die Funktion [6 Punkte]

```
merge :: Ord t => Heap t -> Heap t -> Heap t
```

die zwei linksgerichtete Heaps zu einem einzigen linksgerichteten Heap zusammenfügt.

Beispiel:

```
merge (Node Leaf 0 Leaf) (Node Leaf 1 Leaf) =
       Node (Node Leaf 1 Leaf) 0 Leaf
```

Hinweis:
Beim Zusammenfügen zweier nicht leerer Heaps sollte derjenige mit der größten Wurzel rekursiv mit dem rechten Kind des anderen Heaps zusammengeführt werden.

Beispiellösung:

```
merge :: Ord t => Heap t -> Heap t -> Heap t
merge h Leaf = h
merge Leaf h = h
merge h1@(Node left1 v1 right1) h2@(Node left2 v2 right2)
  | minDepth merged < minDepth other = Node other root merged
  | otherwise = Node merged root other
  where (merged, root, other) = if v1 < v2
                                then (merge right1 h2, v1, left1)
                                else (merge right2 h1, v2, left2)
```

Aufgabe 4 (Lambda) [24 Punkte]

Erinnerung: Die folgenden Regeln definieren wie λ-Abstraktionen vom Funktionstyp eingeführt werden können ($\to$I), und Terme vom Funktionstyp auf andere Terme angewandt werden können ($\to$E), (wobei u eine Variable, M, N Terme, und A, B Typen sind):

$$(\to\mathrm{I})\ \frac{\begin{array}{c}\overline{u : A}\\ \vdots \\ M : B\end{array}}{\lambda u.M : A \to B} \qquad (\to\mathrm{E})\ \frac{M : A \to B \quad N : A}{M\,N : B}$$

(a) Vervollständigen Sie folgende Reduktion, die das Resultat dessen vereinfacht, wie die $\to$E Elimination direkt auf das Resultat der $\to$I Introduktion angewandt wird (wobei $\mathcal{D}$ und $\mathcal{E}$ beliebige Ableitungen sind): [5 Punkte]

$$\to\mathrm{E}\,\frac{\to\mathrm{I}\,\dfrac{\begin{array}{c}\overline{u : A}\\ \mathcal{D}\\ M : B\end{array}}{\lambda u.M : A \to B} \quad \begin{array}{c}\mathcal{E}\\ N : A\end{array}}{(\lambda u.M)\,N : B} \quad \rightsquigarrow \quad \ldots$$

Beispiellösung:

$$\to\mathrm{E}\,\frac{\to\mathrm{I}\,\dfrac{\begin{array}{c}\overline{u : A}\\ \mathcal{D}\\ M : B\end{array}}{\lambda u.M : A \to B} \quad \begin{array}{c}\mathcal{E}\\ N : A\end{array}}{(\lambda u.M)\,N : B} \quad \rightsquigarrow \quad \begin{array}{c}\dfrac{\mathcal{E}}{u : A}\\ \mathcal{D}[u \mapsto N]\\ M[u \mapsto N] : B\end{array}$$

(b) Welches zentrale Prinzip des λ-Kalküls begründet obige Reduktion? [2 Punkte]

Beispiellösung:

Die β-reduktion von λ-Abstraktionen wird durch die obige Beweisreduktion begründet.

Erinnerung: Die folgenden Regeln definieren wie Tupel vom Produkttyp eingeführt werden können ($\times$I) und Terme vom Produkttyp projiziert werden können ($\times\mathrm{E}_1$), (wobei u eine Variable, M, N Terme, und A, B Typen sind):

$$(\times\mathrm{I})\ \frac{M : A \quad N : B}{\langle M, N\rangle : A \times B} \qquad (\times\mathrm{E}_1)\ \frac{M : A \times B}{\mathbf{fst}\, M : A} \qquad (\times\mathrm{E}_2)\ \frac{M : A \times B}{\mathbf{snd}\, M : B}$$

(c) Geben Sie eine passende Reduktion an, die das Resultat dessen vereinfacht, wie die $\times\mathrm{E}_1$ Elimination direkt auf das Resultat der $\times$I Introduktion angewandt wird: [5 Punkte]

Beispiellösung:

$$\times\mathrm{E}_1\,\frac{\times\mathrm{I}\,\dfrac{\overset{\mathcal{D}}{M : A} \quad \overset{\mathcal{E}}{N : B}}{\langle M, N\rangle : A \times B}}{\mathbf{fst}\,\langle M, N\rangle : A} \quad \rightsquigarrow \quad \begin{array}{c}\mathcal{D}\\ M : A\end{array}$$

(d) Welches Prinzip des λ-Kalküls begründet obige Reduktion? [2 Punkte]

Beispiellösung:

Die Projektion von gerade konstruierten Paaren auf eine Koordinate wird durch die obige Beweisreduktion begründet.

Oder: Die zweite Koordinate eines Paares ist für die Projektion auf die erste unnötig.

Erinnerung: Beispielnotation für Konstruktoren und Destruktoren der Programmtypen.

Name	**Konstruktor**	: **Typ**	**Destruktor**
Produkt	$\langle M, N\rangle$	$: A \times B$	$\mathbf{fst}\, M, \mathbf{snd}\, M$
Funktion	$\lambda u.M$	$: A \to B$	$M\, N$
disj. Summe	$\mathbf{inl}\, M, \mathbf{inr}\, M$	$: A + B$	$\mathbf{case}\, M\, \mathbf{of}\, \mathbf{inl}\, u \Rightarrow N \mid \mathbf{inr}\, w \Rightarrow O$
Unit	$\langle\rangle$	$: \mathbf{1}$	
Empty		$: \mathbf{0}$	$\mathbf{abort}\, M$

Für jeden der folgenden Typen geben Sie ein (terminierendes) funktionales Programm als λ-Term diesen Typs an oder begründen Sie *kurz* warum es solch ein Programm nicht gibt.

(e) $A \times B \to B \times A$ [2 Punkte]

Beispiellösung:

$\lambda p.\langle \mathbf{snd}\, p, \mathbf{fst}\, p\rangle$

(f) $A + B \to B + A$ [2 Punkte]

Beispiellösung:

$\lambda e.\mathbf{case}\, e\, \mathbf{of}\, \mathbf{inl}\, a \Rightarrow \mathbf{inr}\, a \mid \mathbf{inr}\, b \Rightarrow \mathbf{inl}\, b$

(g) $A + B \to B \times A$ [2 Punkte]

Beispiellösung:

Das ist unmöglich denn wenn man aus $A+B$ nur ein A hat kann man kein Paar mit einem Term des beliebigen Typs B bilden (was ja auch ein unmögliches empty sein könnte).

(h) $A \to (B \to (A \times B))$ [2 Punkte]

Beispiellösung:

$\lambda a.\lambda b.\langle a, b\rangle$

(i) $(A \to B) \to (A \times B)$ [2 Punkte]

Beispiellösung:

Das ist unmöglich denn wenn man eine Funktion von A nach B gegen hat kann man keinen Term des beliebigen Typs A bilden (was ja auch ein unmögliches empty sein könnte).

Aufgabe 5 (MPI) [9 Punkte]

(a) In dem auf der nächsten Seite gezeigten Code werden Messwerte von Solarmodulen verarbeitet, die die aktuell erzeugte Energie repräsentieren. Die Funktion `avgPowerGenerated` erhält die Messwerte als Array `data` der Länge `length` und berechnet parallel deren Durchschnitt, indem die Berechnung auf mehrere MPI-Prozesse verteilt wird. Sie können davon ausgehen, dass `length` dabei ein Vielfaches von `size` ist. Die Funktion `avg` berechnet sequentiell den Durchschnitt des übergebenen Arrays. [3 Punkte]

Füllen Sie die untenstehenden Tabellen mit den Werten aus, die die angegebenen Variablen nach Ausführung der angegebenen Zeile haben. Zu Beginn der Ausführung liegen die Daten nur auf dem root-Prozess vor, wie in der ersten Tabelle angegeben.

Variable: `data`

Prozess Nr.	**Wert nach Zeile 3**
0	{ 7, 1, 4, 8, 4, 2, 6, 0 }
1-3	-

Variable: `localData`

Prozess Nr.	**Wert nach Zeile 10 (Lösung)**
0	{ 7, 1 }
1	{ 4, 8 }
2	{ 4, 2 }
3	{ 6, 0 }

Variable: `localAvg`

Prozess Nr.	**Wert nach Zeile 12 (Lösung)**
0	4
1	6
2	3
3	3

Variable: `globalSum`

Prozess Nr.	**Wert nach Zeile 16 (Lösung)**
0	16
1	0
2	0
3	0

Beispiellösung: Siehe Tabellen.

```
1  const int ROOT = 0;
2
3  double avgPowerGenerated(double *data, int length) {
4      int size; MPI_Comm_size(MPI_COMM_WORLD, &size);
5
6      int localLength = length / size;
7      double *localData = malloc(localLength * sizeof(double));
8      MPI_Scatter(data, localLength, MPI_DOUBLE,
9                  localData, localLength, MPI_DOUBLE,
10                 ROOT, MPI_COMM_WORLD);
11
12     double localAvg = avg(localData, localLength);
13
14     double globalSum = 0;
15     MPI_Reduce(&localAvg, &globalSum, 1, MPI_DOUBLE,
16             MPI_SUM, ROOT, MPI_COMM_WORLD);
17     return globalSum / size;
18 }
```

(b) Statt den Durchschnitt aller Messwerte zu berechnen, soll nun der **niedrigste Durchschnitt** der Messwerte von Gruppen von Modulen identifiziert werden. Die Solarmodule sind dafür in quadratische, überschneidungsfreie Gruppen einer fixen Größen `N` aufgeteilt. Die Berechnung der Durchschnitte der einzelnen Gruppen soll dabei weiterhin auf die beteiligten MPI-Prozesse verteilt werden. Es soll **keine Vorverarbeitung der Daten durch den Root-Prozess** stattfinden. Die Indizierung der Messwerte der Solarmodule im Array `data` und eine Beispielgruppe sind in der untenstehenden Grafik schematisch dargestellt. [3 Punkte]

Erklären Sie, welche Änderungen zu dem obenstehenden Code nötig sind. Gehen Sie dabei insbesondere darauf ein, welche der in der Vorlesung behandelten **kollektiven MPI-Operationen** dafür verwendet werden können. Sie müssen für die Lösung der Aufgabe keinen Programm-Code schreiben.

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Beispiellösung:

Dadurch, dass die von den Prozessen jeweils benötigten Daten nicht mehr hintereinander im Speicher liegen, können die Daten nicht mehr mit `MPI_Scatter` verteilt werden. Stattdessen kann `MPI_Broadcast` genutzt werden, um die Daten auf die Prozesse zu verteilen.

Hinweis: `MPI_Scatterv` kann nicht ohne weiteres verwendet werden, da die verteilten Speicherbereiche jeweils zusammenhängend sein müssen.

Da nicht mehr der Durchschnitt, sondern das Minimum der in den Prozessen berechneten Werte bestimmt werden soll, muss außerdem die Operation von `MPI_SUM` zu `MPI_MIN` geändert werden.

(c) Nennen Sie je einen Vorteil der Parallelisierungsansätze *shared memory* und *message passing*. Beschreiben Sie außerdem eine Situation, in der der Einsatz von *shared memory* nicht sinnvoll möglich ist. [3 Punkte]

Beispiellösung:

Vorteile *shared memory*

- Keine Datenübertragung zwischen den Prozessen nötig
- Einfachere Umsetzung von *task parallelism* möglich
- Unterstützung diverser Arten von Parallelität (e.g., Streams, Futures)

Vorteile *message passing*

- Keine *race conditions* wegen überschneidender Speicherzugriffe möglich
- Keine *memory inconsistencies* möglich
- Unterstützung von kollektiven Operationen zur einfachen Umsetzung von *data parallelism*

Situationen, in denen *shared memory* nicht angewendet werden kann

- Rechenzentren
- Volunteer-Computing, d.h. das Verteilen von Aufgaben auf viele, meist leistungsschwächere (Privat-)Geräte
- i.Allg. Berechnungen, die auf mehrere Geräte verteilt werden (da kein gemeinsamer Speicher vorhanden)

Aufgabe 6 (Java) [11 Punkte]

(a) Der folgende Code implementiert die Temperatursteuerung eines Smart-Home-Systems. Das System kennt die zwei Betriebsmodi `AUTO` und `MANUAL`, die sich gegenseitig ausschließen. Die Methode `runAutoTemperatureControl` wird jede Minute von einem eigenen Thread aus aufgerufen und berechnet im Modus `AUTO` einen vordefinierten Temperaturverlauf. Durch Aufruf der Methode `setManualTemperature` kann der Modus auf `MANUAL` geändert werden. Dadurch wird die übergebene Temperatur beibehalten und nicht mehr durch den automatisch berechneten Temperaturverlauf verändert. [3 Punkte]

Zu welchem Problem kann es dadurch kommen, dass die beiden Methoden `runAutoTemperatureControl` und `setManualTemperature` aus verschiedenen Threads aufgerufen werden? Nennen Sie einen passenden Java-Mechanismus und beschreiben Sie, wie Sie damit das Problem lösen können.

```java
abstract class SmartHome {
    enum OperationMode {
        AUTO,
        MANUAL
    }

    private int targetTemperature;
    private OperationMode mode;

    public void runAutoTemperatureControl() {
        if (mode == OperationMode.AUTO) {
            targetTemperature = calculateAutoTemperature();
        }
    }
    public void setManualTemperature(int temperature) {
        mode = OperationMode.MANUAL;
        targetTemperature = temperature;
    }
    abstract int calculateAutoTemperature();
}
```

Beispiellösung:

Durch den Aufruf der Methoden aus unterschiedlichen Threads kann es zu *race conditions* kommen. Zum Beispiel könnte in Thread A zuerst die Methode `runAutoTemperatureControl` aufgerufen und die Bedingung `mode == OperationMode.AUTO` zu `true` ausgewertet werden. Danach könnte zu Thread B gewechselt werden, in dem `setManualTemperature` vollständig ausgeführt wird. Danach könnte wieder in Thread A die Variable `targetTemperature` gesetzt werden, obwohl die durch den Nutzer gesetzt Temperatur beibehalten werden soll.

Das Problem lässt sich lösen, indem beide Methoden als `synchronized` deklariert werden. Dadurch werden die beiden Methoden vollständig ausgeführt, bevor ein anderer Thread mit der nächsten Abarbeitung einer der Methoden beginnen kann.

(b) Um Deadlocks durch zirkuläre Abhängigkeiten zu vermeiden, soll ein Synchronisationsmechanismus benutzt werden, der es erlaubt mehrere Ressourcen gleichzeitig zu akquirieren. Die Ressourcen werden dabei durch Objekte des Typs `Object` repräsentiert. Vervollständigen Sie die untenstehende Implementierung, indem Sie *signals* benutzen, um Threads, deren angefragte Objekte nicht zur Verfügung stehen, solange warten zu lassen, bis neue Objekte zur Verfügung stehen. Benutzen Sie zur Synchronisation nur das `this`-Objekt der `ObjectSemaphore`-Klasse. [3 Punkte]

Hinweis: Sie müssen keine Behandlung von Ausnahmen (Exceptions) implementieren. Die Größe der Kästchen steht dabei nicht notwendigerweise im Verhältnis zur Länge der erwarteten Lösung.

Beispiellösung:

```
1  public class ObjectSemaphore {
2      public ObjectSemaphore(Set<Object> objects) {
3          availableObjects = new HashSet<>(objects);
4      }
5      private Set<Object> availableObjects;
6
7      public synchronized void acquireAccessTo(Set<Object> objects) {
8          while (!availableObjects.containsAll(objects)) {
9              wait(); // we should catch the InterruptedException here
10         }
11         availableObjects.removeAll(objects);
12     }
13     public synchronized void releaseAccessTo(Set<Object> objects) {
14         availableObjects.addAll(objects);
15         notifyAll();
16     }
17 }
```

(c) Angenommen ein Thread fragt zwei Ressourcen A und B durch Aufruf der Methode `acquireAccessTo` an und benötigt zu einem späteren Zeitpunkt zusätzlich eine dritte Ressource C. Welche Methodenaufrufe müssen nach dem initialen Aufruf von `acquireAccessTo` auf der Klasse `ObjectSemaphore` ausgeführt werden, um sicherzustellen, dass es nicht zu Deadlocks durch zirkuläre Abhängigkeiten mit anderen Threads kommen kann? Nach den beschriebenen Methodenaufrufen soll der Thread Zugriff auf die Ressourcen A, B und C haben. [2 Punkte]

Sie müssen für die Lösung der Aufgabe keinen Programm-Code schreiben.

Beispiellösung: Um Deadlocks durch zirkuläre Abhängigkeiten zu vermeiden, muss zunächst `releaseAccessTo` für die Ressourcen A und B aufgerufen werden, bevor `acquireAccessTo` für die Ressourcen A, B und C aufgerufen werden kann. So lässt sich die Coffman-Bedingung *hold and wait* vermeiden.

(d) In Java ist es im Allgemeinen nicht sicher, auf Werte zuzugreifen, die von einem anderen Thread gesetzt wurden. Welche Beziehung muss zwischen zwei Anweisungen existieren, damit dies sicher möglich ist? Nennen Sie zwei Möglichkeiten diese Beziehung zwischen Statements, die in unterschiedlichen Threads ausgeführt werden, herzustellen. [3 Punkte]

Beispiellösung: Damit ein Wert, der von einem Statement A in einem Thread X gesetzt wurde, von einem Statement B in einem Thread Y sicher gelesen werden kann, muss zwischen den Statements A und B eine happens-before-Beziehung bestehen. Eine happens-before-Beziehung kann hergestellt werden durch:

- `Thread.start` zwischen Statements vor dem Starten des Threads und Statements in dem gestarteten Thread
- `Thread.join` zwischen Statements in dem Thread, auf dessen Beendigung gewartet wird, und den Statements nach `Thread.join`
- `synchronized` zwischen Statements in aufeinanderfolgenden Ausführungen des Blocks, bzw. der Methode, die als `synchronized` deklariert ist
- `volatile` zwischen aufeinanderfolgenden Zugriffen auf eine als `volatile` deklarierte Variable

Aufgabe 7 (Design by Contract) [10 Punkte]

(a) Im folgenden Codeausschnitt ist die Steuerung eines Roboters implementiert. Um eine vordefinierte Aufgabe zu lösen, wird die Methode `solveTask` aufgerufen, die sicherstellen soll, dass durch den Betrieb des Roboters keine Menschen verletzt werden. Die Methode `operateAutonomously` wird dann verwendet, um einen Schraubenzieher von der Werkbank zu holen und zu einem Auto zu bringen. [4 Punkte]

Nennen Sie die zwei Stellen, an denen ein Methodenvertrag verletzt wird (Vor- / Nachbedingung nicht eingehalten) und erklären Sie, wodurch der Vertrag verletzt wird.

Hinweis: Gehen Sie davon aus, dass `alert` keine Variablenwerte verändert und nach dem Aufruf immer mit der Abarbeitung der aufrufenden Methode fortgefahren wird.

```
1  public class Robot {
2      private boolean humanIsNear;
3      private boolean stopped;
4
5      /*@ requires humanIsNear == false;
6        @ ensures stopped == true;
7        @*/
8      void operateAutonomously() {
9          moveFast(WORKBENCH);
10         grabObject(workbenchHolder, SCREWDRIVER);
11         moveFast(CAR);
12
13         stopped = false;
14     }
15
16     /*@ requires stopped == true;
17       @ ensures stopped == true;
18       @*/
19     void solveTask() {
20         if (humanIsNear) {
21             alert();
22         }
23         operateAutonomously();
24     }
25
26     /* ... */
27 }
```

Beispiellösung:

1. Die Methode `solveTask` verletzt als Aufruferin die Vorbedingung `!humanIsNear` der Methode `operateAutonomously`, da weder die Vorbedingung von `solveTask` fordert, dass `!humanIsNear` gelten muss, noch in der Implementierung der Methode sichergestellt wird, dass `!humanIsNear` gilt.
2. Die Methode `operateAutonomously` verletzt ihre Nachbedingung `stopped`, da die Variable `stopped` in der Implementierung immer auf `false` gesetzt wird.

Hinweis: Die Nachbedingung von `solveTask` ist nicht verletzt, da durch die Nachbedingung von `operateAutonomously` garantiert ist, dass `stopped == true` gilt.

(b) Welches Prinzip, das beim Entwerfen von Methodenverträgen beachtet werden soll, ist im obenstehenden Code verletzt? Beschreiben Sie, welche Auswirkungen die Verletzung des Prinzips auf die Nutzung der Klasse `Robot` hat. [2 Punkte]

Beispiellösung: Im obenstehenden Code ist das Prinzip der *precondition availability* verletzt, das besagt, dass alle Teile der Vorbedingung einer Methode für die Aufruferin sichtbar sein müssen. Dadurch, dass dieses Prinzip nicht eingehalten wird, kann die Klasse `Robot` nicht sicher verwendet werden, da unklar ist, ob die Methoden `operateAutonomously` und `solveTask` tatsächlich das erwartete und spezifizierte Verhalten ausführen werden.

(c) Im nachfolgenden Code ist die Methode `grabObject` dargestellt. Schreiben Sie Vor- und Nachbedingungen, die sicherstellen, dass die Methode das Objekt `target` aus dem Storage `from` in den Storage `hand` überführt. Die Methode `Storage::getContent` gibt dabei entweder den Inhalt eines Storages zurück oder `null`, falls ein Storage leer ist. [4 Punkte]

Beachten Sie dabei, dass ein Storage nicht mehr als ein Objekt gleichzeitig beinhalten kann und stellen Sie sicher, dass ein Objekt nicht in mehr als einem Storage beinhaltet ist.

Beispiellösung:
Hinweis: Da in der Aufgabenstellung die Methode `getContent` inkonsistent definiert wurde, gibt es mehrere korrekte Lösungen, von denen im Folgenden zwei vorgestellt werden.

Lösung mit `getContent`:

```
1  public class Robot {
2      /* ... */
3
4      private Storage hand;
5
6      /*@ requires from.getContent() == target;
7        @ requires hand.getContent() == null;
       entweder:
8        @ ensures hand.getContent() == target;
       oder:
9        @ ensures hand.getContent() ==
10                   \old(from.getContent());
11       @ ensures from.getContent() == null;
12       @*/
13     void grabObject(Storage from, SmallObject target) { /* ... */ }
14 }
```

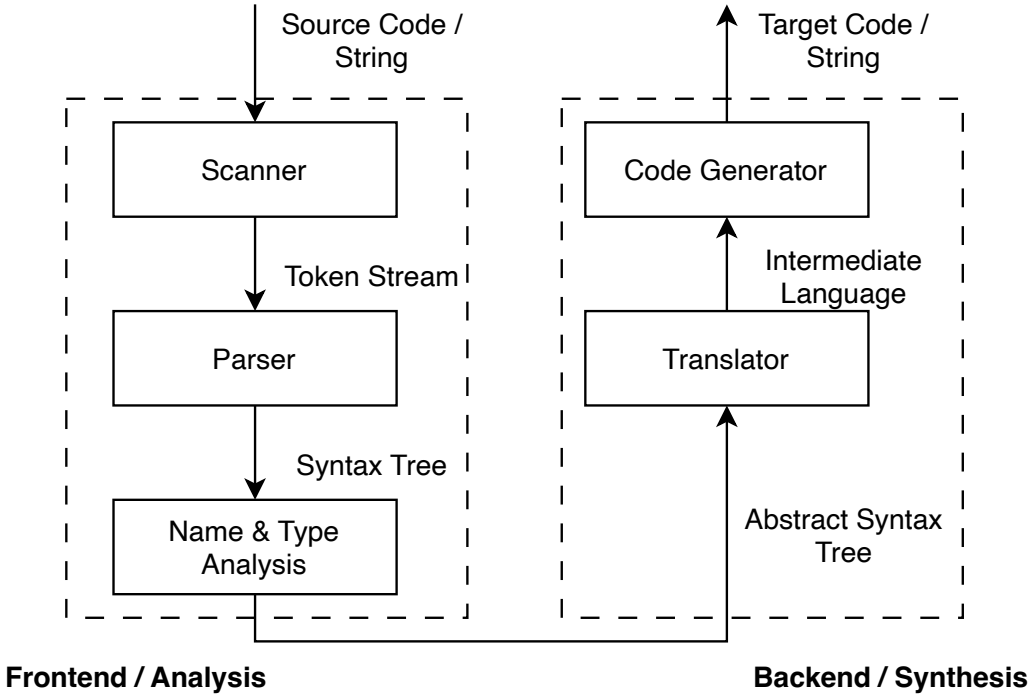
Lösung mit `getContentIfPresent`:

```
1  public class Robot {
2      /* ... */
3
4      private Storage hand;
5
6      /*@ requires from.isContentPresent() &&
7            from.getContentIfPresent() == target;
8        @ requires !hand.isContentPresent();
       entweder:
9        @ ensures hand.isContentPresent() &&
10           hand.getContentIfPresent() == target;
       oder:
11       @ ensures hand.isContentPresent() &&
12           hand.getContentIfPresent() == \old(from.getContentIfPresent());
13       @ ensures !from.isContentPresent();
14       @*/
15     void grabObject(Storage from, SmallObject target) { /* ... */ }
16 }
```

Aufgabe 8 (Compiler) [30 Punkte]

(a) Skizzieren Sie den wesentlichen Aufbau eines Compilers. Zeichnen Sie dabei die verschiedenen Compiler-Bestandteile und die entsprechenden Ein-/Ausgaben ein. Kennzeichnen Sie auch die zwei Ihnen bekannten Compiler-Phasen. [6,5 Punkte]

Beispiellösung:



(b) Geben Sie reguläre Ausdrücke an, die die folgende(n) Sprache(n) beschreiben. [8 Punkte]

- Die Menge aller Wörter $w \in \{a, b, c, d\}^*$, bei denen jedem a mindestens drei b vorangehen. (2P)

  Beispiellösung:

  [bbba|b|c|d]*

- Die Sprache aller E-Mail-Adressen, die vor dem @ beliebig Buchstaben und Zahlen haben kann (mindestens jedoch ein Zeichen), die Domains ausschließlich aus Buchstaben bestehen, und die Länderspezifische Domain aus 2 oder 3 Kleinbuchstaben besteht. (3P)

  Beispiellösung:

  [a-zA-Z0-9]+@[a-zA-Z].[a-z][a-z][a-z]?

- Die Sprache aller gültigen Zahlen in wissenschaftlicher Notation (Exponentialschreibweise). (3P) Hinweis:

  – Sowohl die Zahl selbst als auch der Exponent kann positiv oder negativ sein.
  – Der Exponent muss mit e oder E beginnen.
  – Eine Dezimalstelle ist optional (z. B. 2E8 ist auch gültig).

  Beispiellösung:

  -?[0-9]+(.[0-9]+)?[eE][-+]?[0-9]+

(c) Gegeben die folgende Grammatik: [1 Punkt]

$$S \to aSa$$

$$S \to bSb$$

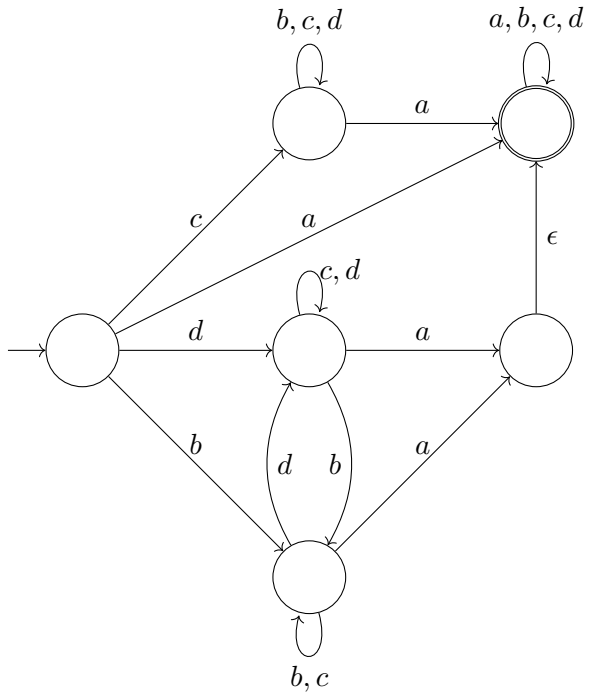
$$S \to \varepsilon$$

Welche Sprache beschreibt diese Grammatik?

Beispiellösung:

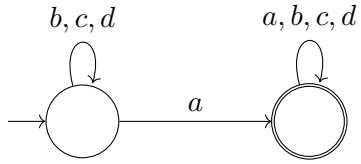
Die Sprache aller Palindrome über das Alphabet $\{a, b\}$.

(d) Welche Sprache beschreibt der folgende nicht-reduzierte Automat? Zeichnen Sie einen minimalen Automaten, der die gleiche Sprache akzeptiert. [3 Punkte]



Beispiellösung:

Äquivalenter reduzierter Automat:



Damit ist das die Sprache aller Wörter über das Alphabet $\{a, b, c, d\}$, die mindestens ein a enthalten.

(e) Diskutieren Sie anhand von zwei Eigenschaften welche Auswirkungen die Entscheidung beim Sprachdesign zwischen einer getypten und einer nicht getypten Sprache auf den Compiler der Sprache hat. [2 Punkte]

Beispiellösung:

Getypt:

- Typüberprüfung zu Compile-Zeit
- Fehlererkennung beim Kompilieren
- Bessere Optimierung
- Komplexere Kompiler
- effizientere Speicherverwaltung

Ungetypt:

- Typüberprüfung zu Laufzeit
- Fehlererkennung erst bei Laufzeit (e.g., TypeErrors)

(f) Wann ist ein Programm typsicher? [1 Punkt]

Beispiellösung:

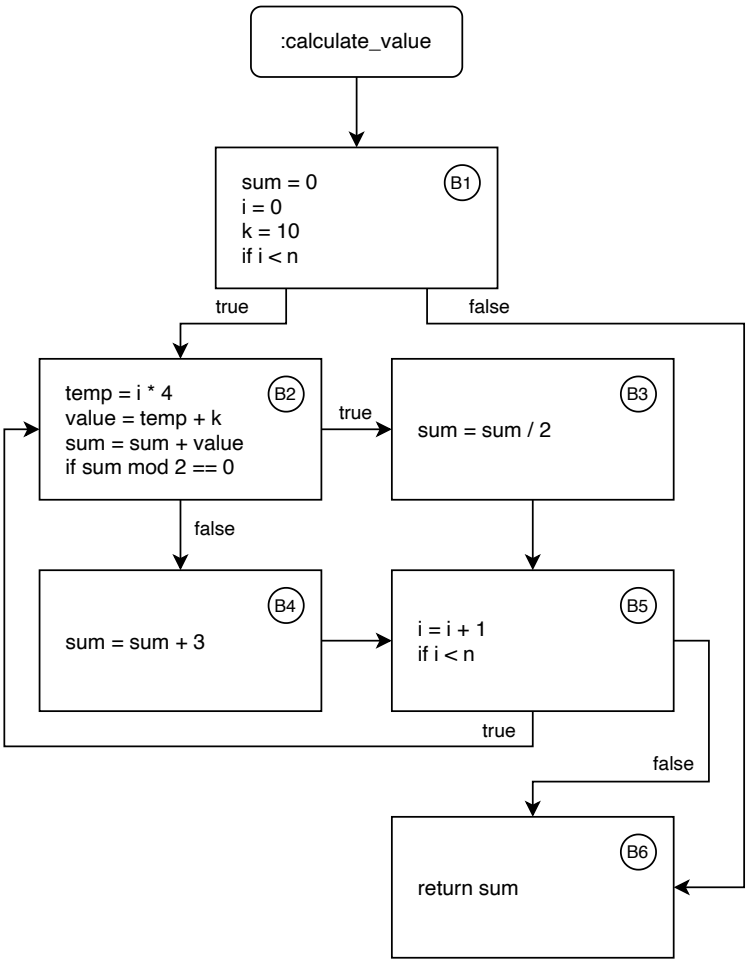
Wenn seine Ausführung nicht zu Typfehlern führt. ODER Wenn es typkorrekt und in einer stark getypten Sprache geschrieben ist. (Typkorrektheit impliziert Typsicherheit in einer stark getypten Sprache.)

(g) Zeichnen Sie *Basics Blocks* entsprechend dem folgenden Pseudocode. Zeichnen Sie dabei größtmögliche zusammenhänge Blöcke. Eingabeparameter einer Funktion müssen nicht extra deklariert werden. [4,5 Punkte]

```
function calculate_value(n)
sum = 0
i = 0
k = 10
while i < n do
    temp = i * 4
    value = temp + k
    sum = sum + value
    if sum mod 2 == 0 then
        sum = sum / 2
    else
        sum = sum + 3
    end if
    i = i + 1
end while
return sum
end function
```

Beispiellösung:

TBD figure placement



(h) Wenden Sie drei verschiede aus der Vorlesung bekannte Optimierungen exemplarisch auf den folgenden Pseudocode an. [3 Punkte]

```
function calc(int x, int y){
    return (2*x)+y
}


int i = 0
int k = 5
int s =  i + k
int runs = 3
while i < runs:
    k = 8
    s = s + calc(i, k)
    i = i + 1

print(s)
```

Beispiellösung:

Beispielsweise:

Constant Propagation

```
int s =  0 + 5
```

Constant Folding

```
int s =  5
```

Strength Reduction

```
(2*x) $\rightarrow$ x+x | x << 1
```

Inline Expansion

```
s = s + (2*i)+k
```

Loop unrolling

```
k = 8
s = s + calc(i, k)
i = i + 1

k = 8
s = s + calc(i, k)
i = i + 1

...
```

Dead Code Elimination

```
k = 8
s = s + calc(i, k)
i = i + 1

s = s + calc(i, k)
i = i + 1
```

```
...
```

Code Motion

```
k = 8
while i < runs:
    ...
```

Wahrscheinlich nicht anwendbar: Copy Propagation, Common Subexpression Elimination, Algebraic Optimization

(i) Was sind die Aufgaben eines Compilers bei der Codegenerierung? [1 Punkt]

Beispiellösung:

Übersetzung der Zwischendarstellung in Maschinencode; Registerzuweisung und Speicherverwaltung; Operations- / Instruktionsauswahl; Instruktionsanordnung; Optimierungen für die Zielplattform; Erzeugung von Debug-Informationen und Metadaten; Primitive Datentypen verarbeiten; Datentypen, Arrays, Klassen, und sprachspezifische Konstrukte übersetzen